소프트웨어 프로젝트 최종보고서

작 품 명	**Sort Comparison Visualizer**		
개발기간	2026년 10월 08일 ~ 2026년 11월 02일		
지도교수	컴퓨터공학전공		윤 현 주
구분	학년	학번	성명
개발자	2	20231055	정유운

소프트웨어프로젝트 과목에서 수행한 프로젝트 최종보고서를 첨부와 같이 제출합니다.

2026년 11월 05일

국립금오공과대학교 컴퓨터공학부 컴퓨터공학전공

프로젝트 계획서

Sort Comparison Visualizer

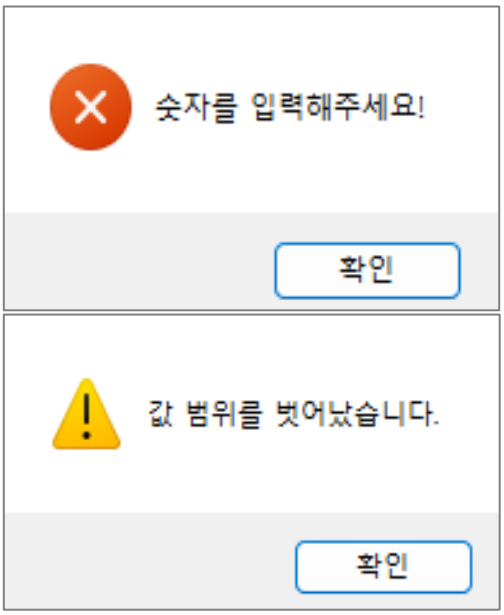
1. 개요

1-(1) 프로젝트의 목표

삽입 정렬, 퀵 정렬, 버블 정렬을 실시간으로 화면에 보여줌으로써 정렬 알고리즘을 배우는 학생들의 이해를 돕게 한다.

1-(2) 결과 개요





(1) 정렬 옵션 지정

값 범위와 데이터의 개수를 지정하여 정렬 데이터를 구성하고 랜덤, 정렬, 역정렬 버튼을 통하여 데이터의 상태를 구성한다.

사용자가 직접 데이터를 입력할 수 있어 유동적인 데이터의 구성이 가능하다.

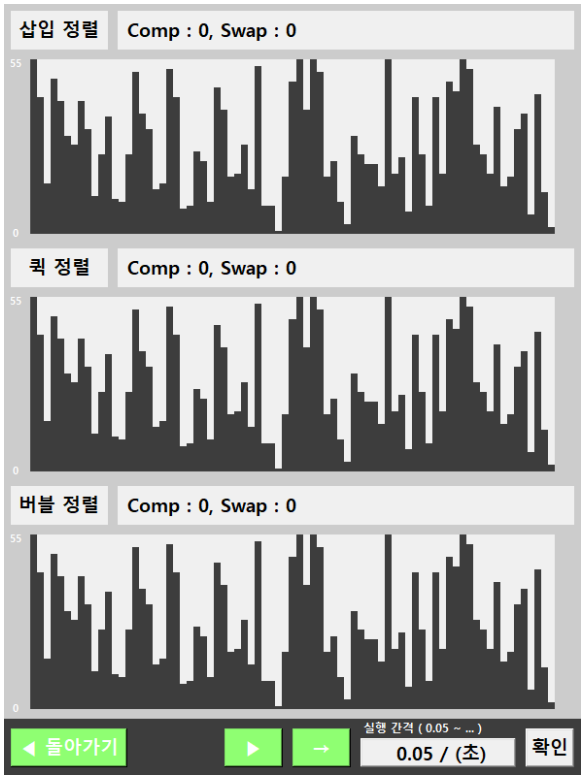
실행 버튼을 눌러 비주얼라이저를 불러온다.

(1-1) 예외 처리

정렬 속도, 값 범위 등 프로그램의 옵션을 수정할 때 예외가 발생하는 경우 유형에 따른 메시지 출력과 함께 데이터를 이전 값으로 되돌린다.

- 입력한 값이 데이터의 범위에 속하지 않는 경우 -> (값 범위를 벗어났습니다.)

- 입력한 값이 데이터의 타입에 맞지 않는 경우 -> (숫자를 입력해주세요!)



(2) 비주얼라이저 초기 구성

이벤트를 통한 데이터의 전달을 통해 초기 페이지를 구성한다. 정렬의 구성은 삽입 정렬, 퀵 정렬, 버블 정렬로 이루어진다.

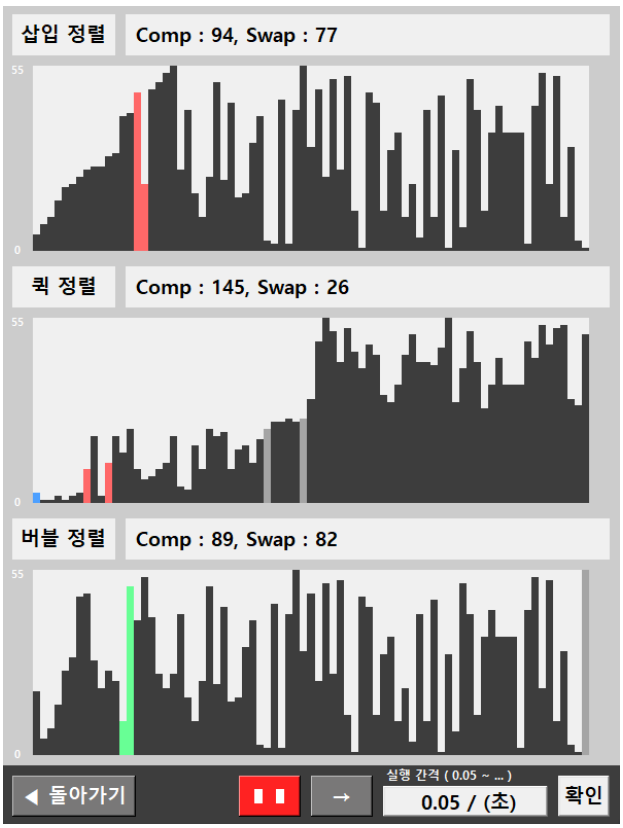
사용자는 버튼을 통하여 비주얼라이저에 대하여 아래와 같은 기능을 수행 할 수 있다.

1. 정렬 시작, 중지 (playBtn, stopBtn)

2. 정렬 처리 1회 실행 (stepBtn)

3. 정렬 속도 조절 (stepBtn, speedAdjBtn)

4. 설정으로 돌아가기 (backBtn)



(3) 비주얼라이저 실행

사용자가 정렬을 처리하는 기능을 요청한 경우 처리 모듈에서 각 정렬에 대한 처리를 수행 한 후 해당 처리에 대한 데이터를 반환하여 그것을 GUI에 반영한다. 전달되는 데이터의 유형은 아래 예시와 같다.

Ex) 정렬 처리 데이터 전달 예시

[[“comp”, 13, 5], [“swap”, 13, 5]]

사용자는 정지 버튼을 통하여 정렬을 중지할 수 있으며 정렬 처리 기능 작동 상태를 불문하고 정렬 처리의 속도를 조절하여 실시간으로 데이터의 변형을 사용자에 맞게 확인이 가능하다.

<table>
<tr>
<td> 
</td>
<td>(4) 비주얼라이저 종료

하나의 정렬 프로세스가 종료하는 경우 모든 막대 그래프가 초록색으로 바뀌면서 종료했다는 것을 사용자에게 보여준다.

Comp, Swap는 정렬에서 어떤 기능을 처리하였는지 보여주는 지표로 Comp는 비교 횟수, Swap은 두 값을 바꾼 횟수를 의미한다.

각 지표를 보고 정렬에 대한 이론상 시간 복잡도를 비교하면 된다.

초기 테스트 결과 기본적으로 해당 정렬들에 대하여 통용적으로 알려진 시간 순서대로 정렬을 완료하였다.

(퀵 정렬 < 삽입 정렬 < 버블 정렬)</td>
</tr>
</table>

2. 요구사항 및 개발환경

2-(1) 기능 요구사항

ID	Title	Description	Priority
REQ-001	시험 데이터 설정	- 시험할 데이터의 유형을 선택하고 이에 따른 속성의 편집을 할 수 있어야 한다. (1) 무작위 데이터 - 값의 범위, 데이터의 개수 (2) 정렬, 역 정렬 데이터 - 데이터의 개수	상
REQ-002	정렬 알고리즘 수행	- 주어진 데이터를 기반으로 3개의 정렬 알고리즘을 수행하고 해당 처리에 대하여 GUI에 자동으로 반영 해야 한다. - 정렬 종류 : 삽입 정렬, 퀵 정렬, 버블 정렬	상
REQ-003	알고리즘 스텝	- 사용자가 직접 한 단계씩 정렬 알고리즘 처리를 볼 수	중

		있어야 한다	
REQ-004	속도 조절	- 알고리즘 수행 전 혹은 수행 도중 수행 속도를 조절할 수 있어야 한다.	중

2-(2) 비기능 요구사항

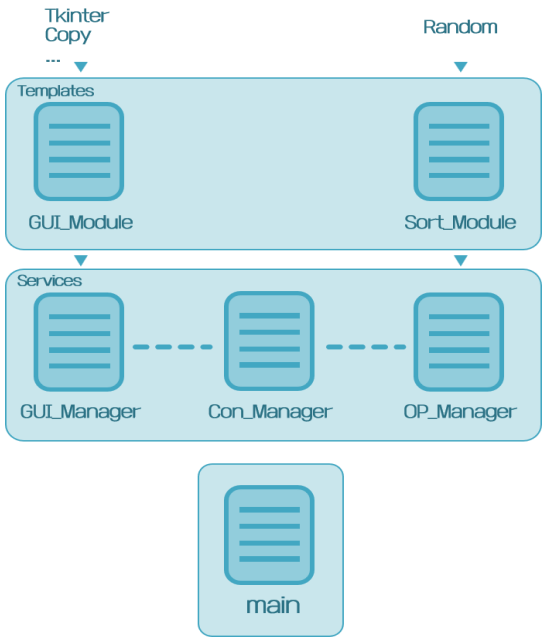
Category	Objective	Architecture Design Methods	Measurement
성능	설정된 알고리즘 실행 속도에 가까운가?	GUI 구현에 쓸모 없는 코드를 배제한다. 최대한 코드를 최적화하여 GPU가 처리를 원활하게 하도록 한다.	스톱워치
가시성	데이터의 정렬이 확실하게 보여지는가?	비교 처리는 두 데이터를 담당하는 GUI 막대를 붉은색으로, 스왑 처리는 초록색으로 바꾼다.	GUI 화면 확인
무결성	기능 처리를 실패하는 경우 해당 데이터를 반영하지 않고 원래 상태로 돌아오는가?	예외 발생 시 GUI의 형태의 따라 이전 데이터의 값으로 그래픽을 맞춘다.	각종 예외 상황을 의도적으로 발생

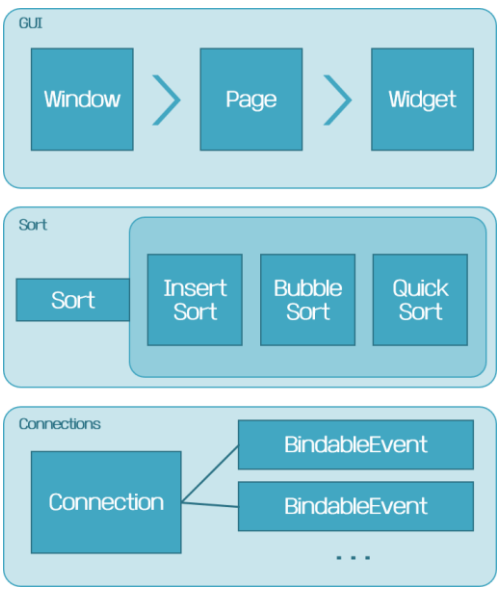
2-(3) 개발 환경

Category	Name	Description	Other
사용 언어	Python 3.14	문제 1에 사용하였던 Tkinter를 그대로 활용하기 위하여 선택	
개발 환경	Visual Studio Code	기존 문제 1에 사용하였던 개발 환경을 유지한다	
제출물	GitHub	GitHub를 통하여 문서 및 소스파일을 저장한다.	Link

3. 구현

3-(1) 시스템 아키텍처





(1) Main.py

Services 폴더 내 Con_Manager를 제외한 Manager 모듈을 전부 불러와 각 서비스에 대하여 초기화를 실시하며, 이후 초기 GUI를 불러온다.

(2) Services

- GUI_Manager

GUI 객체를 생성하고 관리하는 역할을 수행한다. 주로 OP_Manager로부터 들어온 데이터를 통해 정렬 그래프를 반영한다.

- OP_Manager

GUI_Manager로부터 들어온 값으로 초기 정렬 데이터를 생성하고, 이후 정렬 처리 기능을 통하여 해당 처리 사항을 GUI로 전달한다.

- Con_Manager

모듈간의 원할한 독립을 위하여 데이터 전달 기능을 제공한다.

(2)-1 Templates

- GUI_Module

Tkinter 라이브러리를 상속한 모듈로 GUI_Manager가 사용할 클래스들을 정의한다.

- Sort_Module

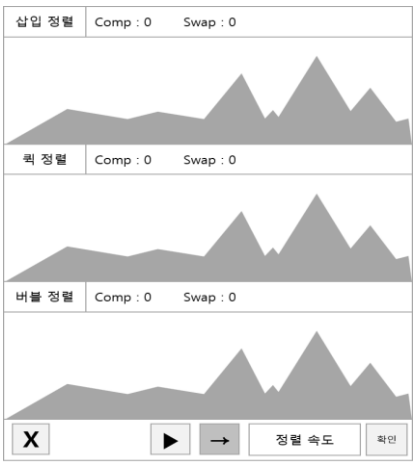
OP_Manager가 사용할 정렬 클래스들을 정의한다.

3-(2) 사용자 인터페이스

1. SettingPage (초기 페이지)



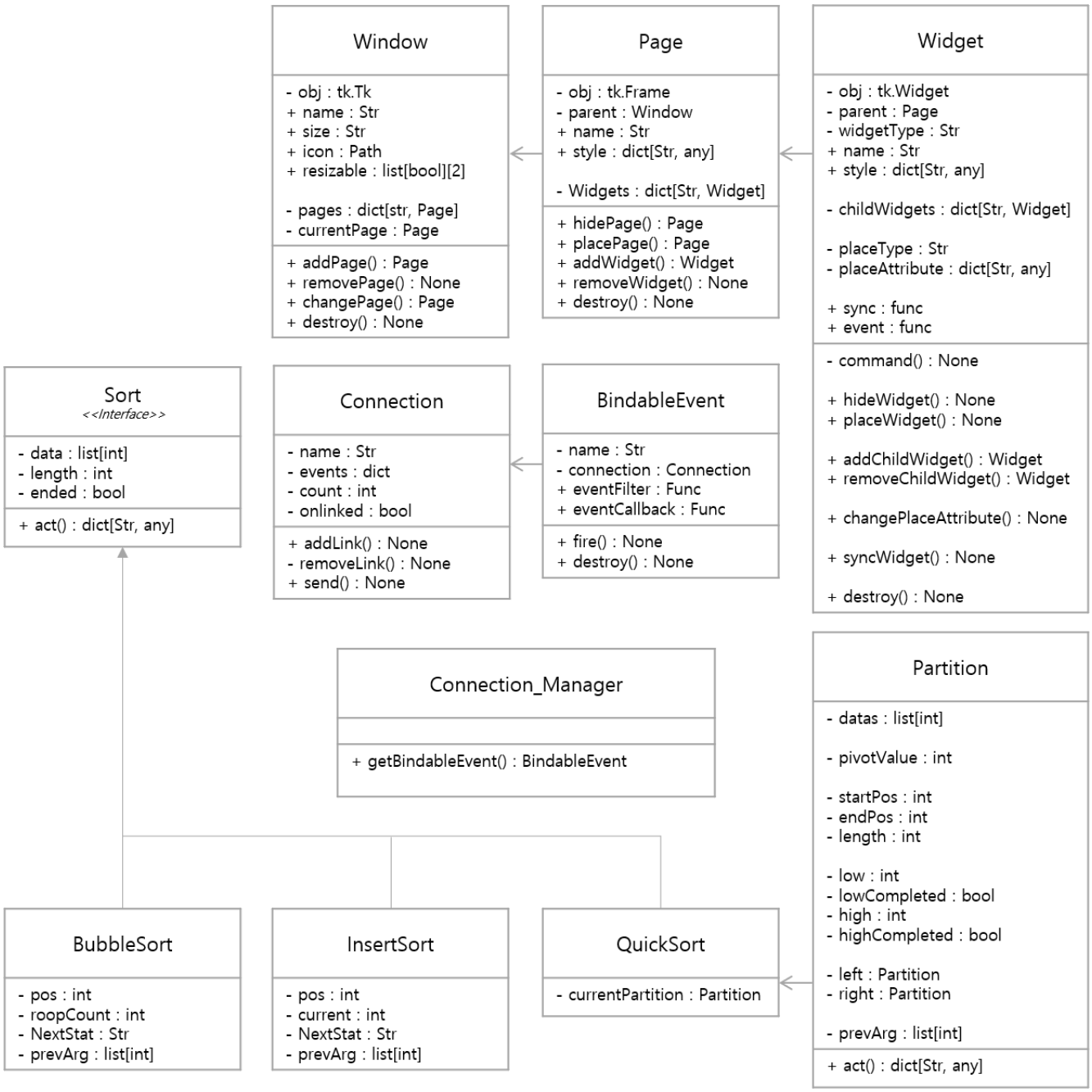
2. VisualPage



ID	Name	Role (Admin/Normal)	Description	Etc
Page-001	SettingPage (Init GUI)	Normal	**1. 시험 데이터 설정** - 무작위 값의 범위, 데이터의 개수를 조정한다 **2. 정렬 시작 버튼** - 입력된 데이터를 바탕으로 Visualizer 페이지로 변경한다	
Page-002	Visualizer	Normal	**1. 정렬 실행, 정지 버튼** - 정렬 알고리즘을 반복 실시 및 정지를 수행한다. **2. 정렬 속도 조절 버튼** - 정렬 알고리즘의 속도를 조절한다 **3. 스텝 버튼** - 정렬 알고리즘을 한 단계 실시한다 **4. 돌아가기 버튼** - Main 페이지로 변경한다	

3-(3) 핵심 클래스

[1] 클래스 다이어그램



[2] 클래스 세부사항

ID	Name	Description	Depend
Cls-001-01	Window	**tk.Tk 객체를 실 객체로 지정하여 감싸는 클래스로 하위 객체인 Page를 관리한다.** **< 주요 변수 >** - obj : 감싸고 있는 tk.Tk 객체 - pages : 소유한 페이지 - currentPage : 현재 실행 중인 페이지 **< 주요 함수 >** - addPage() : 페이지를 추가한다 - changePage() : 현재 페이지를 변경한다	**tk.Tk**
Cls-001-02	Page	**tk.Frame 객체를 실 객체로 지정하여 감싸는 클래스로 하위 객체인 Widget를 관리한다.** **< 주요 변수 >** - obj : 감싸고 있는 tk.Frame 객체 - parent : 해당 페이지를 관리하는 Window 객체 - widgets : 소유한 위젯 - style : 페이지의 기본 스타일 **< 주요 함수 >** - hidePage() : 해당 페이지를 보이지 않게 한다 - placePage() : 해당 페이지를 보이게 한다 - addWidget() : 위젯을 추가한다 - removeWidget() : 위젯을 지운다	**Window** **tk.Frame**
Cls-001-03	Widget	**tk.Widget 객체를 실 객체로 지정하여 감싸는 최하위 클래스로 자식으로 다른 Widget을 지정할 수 있다.** **< 주요 변수 >** - obj : 감싸고 있는 tk.Widget 객체 - parent : 해당 페이지를 관리하는 Page 혹은 Widget 객체 - childWidgets : 소유한 자식 위젯 - style : 페이지의 기본 스타일	**Page** **Widget** **tk.Widget**

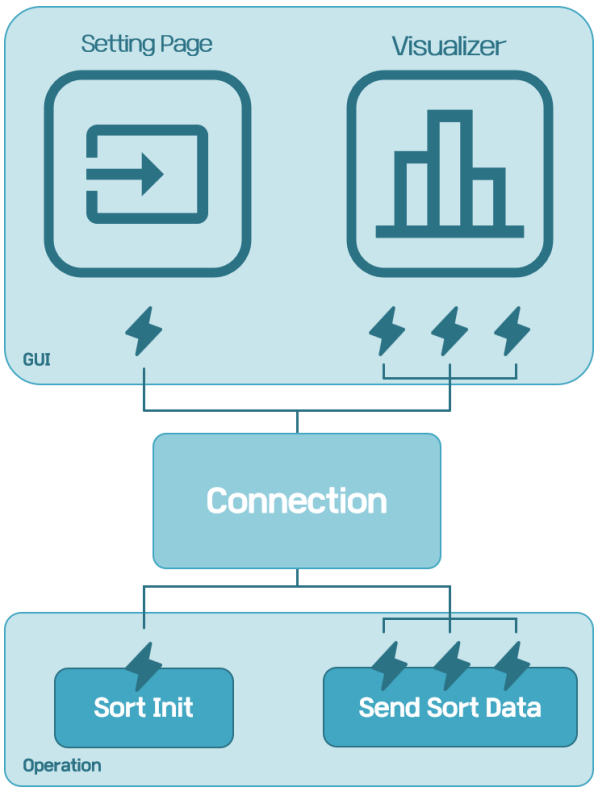
		- sync : 동기화 함수 - event : 이벤트 함수 < **주요 함수** > - command() : sync 혹은 event 함수를 실행한다 - hideWidget() : 위젯을 숨긴다 - placeWidget() : 위젯을 활성화한다 - addChildWidget() : 자식 위젯을 추가한다 - removeChildWidget() : 자식 위젯을 지운다 - changePlaceAttribute() : 위젯의 배치를 변경한다. - syncWidget() : 위젯을 동기화한다. (sync함수 사용)	
Cls-002-00	**Sort**	**정렬 클래스의 인터페이스로 기본적으로 act()라는 함수로 정렬 알고리즘의 로직 처리 기록을 반환해준다.** < **주요 변수** > - data : 숫자 데이터 리스트 - length : datas의 길이 - ended : 정렬 종료 여부 < **주요 함수** > - act() : 아래 서술할 자식 클래스가 오버라이딩을 해야 할 함수로 반환 값은 GUI에 반영할 처리 기록이어야 한다.	**-**
Cls-002-01	**InsertSort**	**삽입 정렬에 대한 클래스다. 삽입 정렬 알고리즘을 수행한다.** < **주요 변수** > (+ Sort 맴버 변수) - pos : 삽입 정렬의 비교를 시작할 위치 - current : 현재 비교 인덱스 위치 - NextStat : 다음 연산을 비교로 할 지 교환으로 할 지 결정한다. (처음에는 비교로 지정한다) - prevArg : 그래프의 상태를 원래대로 돌릴 인덱스를 한데 모아 놓는다. < **주요 함수** > - act(): NextStat에 따라 비교, 교환 연산을 실시한다, 비교 : 현재 데이터에서 두 인덱스(current, current – 1)를 비교하여 data[current]의 값이 더 작은 경우 NextStat를 swap으로 변경한다. 아닌 경우 current의 값을 1 줄인다.	**Sort**

		이후 처리 기록을 추가한다. 교환 : 두 인덱스를 교환한 후 pos의 값을 1을 더하고 current의 값을 pos로 지정하여 비교 연산을 다시 시작할 수 있도록 한다. pos+1가 length보다 큰 경우 해당 정렬이 종료된 것으로 판단한다.	
Cls-002-02	**QuickSort**	**퀵 정렬에 대한 클래스다. 퀵 정렬 알고리즘을 수행한다.** < **주요 변수** > - currentPartition : 현재 정렬이 진행 중인 부분 데이터 집합 < **주요 함수** > - act() : currentPartition의 act()의 값을 받는다. 현재 파티션의 정렬이 종료 되었을 때 양쪽의 파티션이 존재하지 않는 경우 해당 정렬이 종료된 것으로 판단한다.	**Sort**
Cls-002-02-01	**Partition**	**퀵 정렬의 부분 데이터 집합이며 처음에는 모든 범위가 Partition에 존재하다가 정렬 완료 후 분리하면서 left, right 변수를 통해 각 Partition 객체를 연결한다.** < **주요 변수** > - pivotValue : 피벗의 값 - startPos : 파티션의 시작점 - endPos : 파티션의 종료점 - length : 파티션의 길이 - low : 피벗보다 높은 값을 찾는 인덱스 - high : 피벗보다 낮은 값을 찾는 인덱스 - lowCompleted : low가 피벗보다 높은 값 식별 여부 - highCompleted : high가 피벗보다 낮은 값 식별 여부 - left : 왼쪽 파티션 - right : 오른쪽 파티션 - prevArg : 앞서 언급되었던 prevArg와 동일한 역할을 수행한다. < **주요 함수** > - act() : 파티션의 크기에 따라 처리가 구분된다 크기 1 : 해당 파티션은 정렬이 완료되었다고 판단한다 크기 2 : 서로 비교하여 비교 및 교환 연산을 수행하고 정렬이 완료되었다고 판단한다. 크기 3 이상 : 피벗값을 시작 위치의 값으로 지정한 후 low가	**QuickSort**

		먼저 피벗값 보다 큰 값의 인덱스를 찾고 이후 high가 피벗보다 작은 인덱스를 찾는다. 각 인덱스가 멈췄을 때 위치가 엇갈리지 않은 경우 두 요소를 교환하며 그렇지 않은 경우 high와 pivot을 바꾸어 파티션을 분리한다. 예외 상황 1 : pivot값이 최대 값일 때 ㄴ low와 pivot값을 바꾸고 파티션을 분리한다 예외 상황 2 : pivot값이 최소 값일 때 ㄴ 현재 피벗이 정렬이 완료되었다고 판단하고, 피벗을 다음 인덱스로 지정한 후 다시 퀵 정렬 처리를 시작한다.	
Cls-002-03	**BubbleSort**	**버블 정렬에 대한 클래스다. 버블 정렬 알고리즘을 수행한다**. < **주요 변수** > (+ Sort 맴버 변수) - pos : 버블 정렬의 현재 위치 - roopCount : 버블 정렬의 위치 지정 완료 횟수 - NextStat : InsertSort와 동일 - prevArgs : InsertSort와 동일 < **주요 함수** > - act() : NextStat에 따라 비교, 교환 연산을 실시한다 비교 : 현재 데이터에서 두 인덱스(pos, pos + 1)를 비교하여 data[pos]의 값이 더 큰 경우 NextStat를 swap으로 변경한다. 아닌 경우 pos 의 값을 1 높인다. pos가 (length – roopCount) 만큼 간 경우 roopCount를 1 높이고 pos의 값을 0으로 초기화한다. roopCount가 지정된 데이터의 length만큼 돌려질 때 까지 해당 함수를 실행한다.	**Sort**
Cls-003-00	**Connection Manager**	**Con_Manager의 메인 클래스이며 모듈에 고유한 이벤트를 할당해주는 역할을 수행한다.** < **주요 함수** > **getBindableEvent(name) : BindableEvent를 생성하며 Connection에 해당 이름이 있는 객체가 없는 경우 Connection또한 생성하여 생성된 이벤트를 저장시킨 후 이벤트를 반환한다.**	
Cls-003-01	**BindableEvent**	**이벤트 클래스로 해당 객체의 필터, 콜백 함수를 지정하여 같은 커넥션의 다른 이벤트로부터 전달을 받은 경우 두 함수를 실행한다.** < **주요 변수** >	**Secrets Connection**

		- name : 처음 getBindableEvent()의 매게변수인 name과 랜덤으로 고유한 랜덤 값인 secrets.token_hex()를 합쳐 그 둘의 해시 값을 초기 값으로 가지며 이후 사용자가 임의로 이름을 변경할 수 있다. - connection : 이벤트와 연결된 Connection 객체 - eventFilter : 필터링 함수, 작성 시 매게변수로 호출 이벤트의 이름 및 데이터를 반드시 지정하고 반환값은 bool로 지정해야한다. - eventCallback : 콜백 함수, 필터링 함수가 True일 때 작동하며 작성 시 매게변수로 호출 이벤트의 이름 및 데이터를 반드시 지정해야한다. < **주요 함수** > - fire(data) : Connection.send()를 통하여 데이터를 전송한다	
Cls-003-02	**Connection**	**이벤트 객체와 해당 필터, 콜백 함수를 실제로 저장하고 있는 클래스이다. BindableEvent가 fire()를 실행한 경우 다른 이벤트로 데이터를 전달하게 해준다.** < **주요 변수** > - name : Connection_Manager. getBindableEvent() 에서 받은 name 그대로 해당 객체의 이름이 된다. - events : 딕셔너리로써 해당 객체가 소유한 이벤트를 키로 삼아 값에 {“eventFilter” : 필터링 함수, “eventCallback” : 콜백 함수 } 가 포함된다. - count : 이벤트 개수 - onlinked : 데이터 전달 활성화 여부로 count가 2개 이상이면 true 아니면 false다. < **주요 함수** > - addLink() : 이벤트를 추가한다 - removeLink() : 이벤트를 제거한다 - send() : 호출자의 이름과 다른 이벤트의 필터링 함수와 콜백 함수를 수행한다.	

3-(4) 핵심 알고리즘



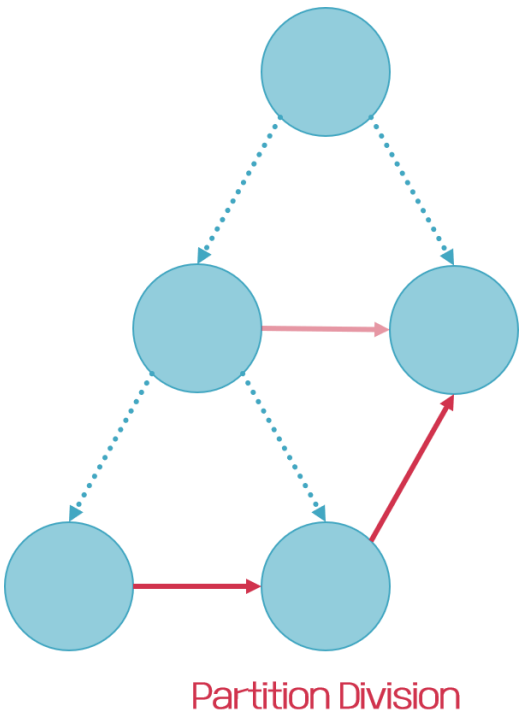
(1) 핵심 프로세스 흐름

1. GUI에서 버튼 이벤트를 누르는 경우 각 버튼의 이벤트 함수가 작동하여 BindableEvent.fire()를 호출함

2. 초기 페이지를 불러오는 경우 처리 모듈인 OP_Manager는 GUI로 부터 받아온 데이터를 통하여 실험 데이터를 생성한다. 이후 해당 데이터를 다시 GUI에 전달하여 초기 Visualizer 화면으로 변경함

3. 정렬 처리를 하는 버튼을 작동시키는 경우 처리 모듈에서 각 정렬에 대해 데이터를 처리하고 난 후 처리 내용을 다시 GUI에 전달한다

4. GUI는 받아온 내용을 통하여 비주얼라이저에 반영한다.



(2) 가장 힘들었던 알고리즘

```
self._left = Partition( self._data, self._startPos, self._high - 1)

self._left._right = Partition( self._data, self._endPos,self._endPos )

if self._right != None:

    self._left._right._right = self._right
```

어떻게 하면 파티션을 올바르게 분리할 수 있을까 생각해보다가 처음에 분리해 놓고 왼쪽과 그 왼쪽의 우측 파티션을 만든 후 왼쪽 파티션의 우측 부분을 지정하면 되는 부분이었다. 추후 분리할 때 부모의 오른쪽에 파티션이 존재하는 경우 왼쪽 파티션의 우측을 부모의 우측 파티션으로 지정하면서 파티션의 연결을 유지시킨다.

4. 실험 및 분석

4-(1) 테스트 방식 및 테스트 데이터

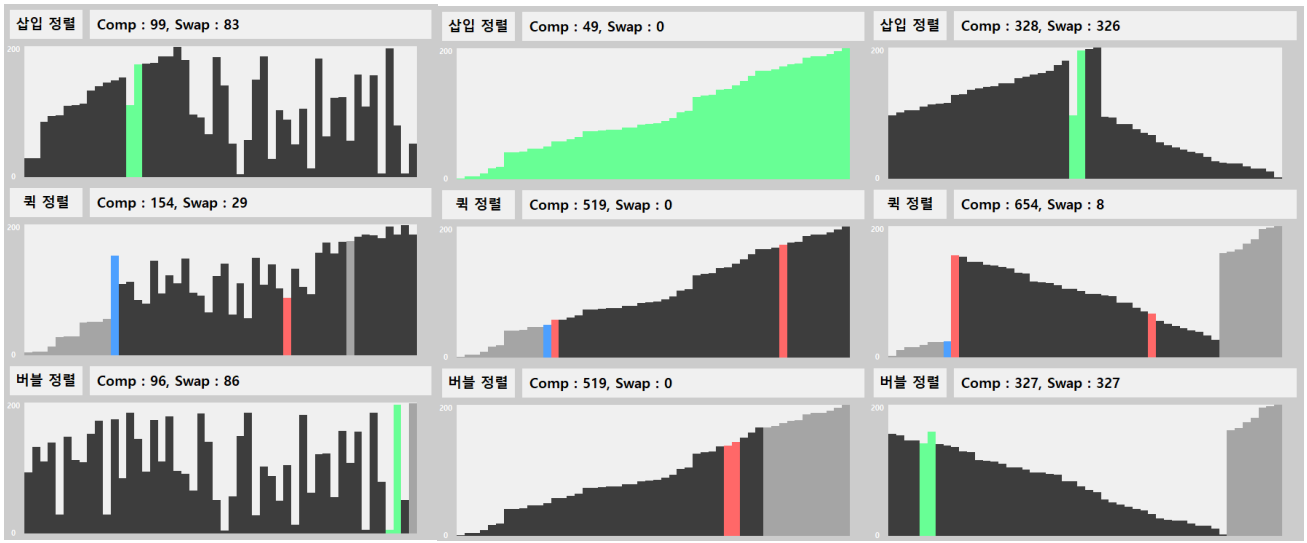
1. 각 데이터의 종류 (랜덤, 정렬, 역정렬) 별로 5개씩 실험을 실시한다

2. 데이터의 값 범위는 [1~200]까지, 데이터의 개수는 [50개]로 제한한다.

3. 테스트 데이터 및 세부 결과는 프로젝트 파일 내 “[문제 2] 문서 기록”에서 “[문제 2] 실험 결과”에서 확인이 가능하다.

(링크 : Link)

4-(2) 기능 테스트 결과



[1. 랜덤 데이터] **[2. 정렬 상태]** **[3. 역정렬 상태]**

모든 데이터 유형에 대하여 정렬이 정상적으로 이루어 졌음을 확인하였다.

4-(3) 비기능 요소 실험 항목 및 실험 결과

Category	Objective	Result	Measurement
성능	- 설정된 알고리즘 실행 속도에 가까운가?	그래프 초기 로딩 시간 (데이터 개수 150개 기준) 1. 0.21초 / 2. 0.40초 / 3. 0.33초 다음 정렬까지의 시간 1. 1.13초 / 2. 1초 / 3. 0.95(?)초 결론 : 다음 정렬 까지의 시간은 위와 같지만 실행 간격을 0.01초로 하였을 때 0.05초와 달리 엄청난 속도로 정렬을 실행하였으므로 사실상 실행 속도는 문제가 없다고 보면 될 것 같다.	그래프 초기 로딩 시간, 정렬 시간을 1초로 지정 후 다음 정렬 그래프 반영까지 걸린 시간 기록
가시성	- 데이터의 정렬이 확실하게 보여지는가?	각 정렬에 대하여 피벗의 위치, 그리고 정렬이 끝난 그래프 바를 볼 수 있어 다음 정렬 위치가 어디인지 짐작할 수 있어서 좋았다.	개발자 결정 및 교수님의 평가
무결성	- 기능 처리를 실패하는 경우 해당 데이터를 반영하지 않고 원래 상태로 돌아오는가?	스탭 버튼을 오토마우스로 눌러본 결과 놀랍게도 아무런 문제가 없었으며 입력 값을 받는 위젯의 경우 설계서에 나온 예외 처리대로 잘 작동하였다.	각종 예외 상황을 의도적으로 발생

5. 결 론

이번 프로젝트는 삽입 정렬, 퀵 정렬, 버블 정렬을 시각적으로 보여주는 비주얼 라이저로써 동일한 데이터에 대하여 각 정렬이 시간 복잡도에 맞게 작동하는 모습을 확인할 수가 있었다.

무작위 데이터의 경우 (퀵 정렬 > 삽입 정렬 > 버블 정렬)

정렬된 데이터의 경우 (삽입 정렬 > 버블 정렬 ≒ 퀵 정렬)

역정렬된 데이터의 경우(퀵 정렬 > 삽입 정렬 = 버블 정렬)

이 시간 면에서 효율적이라는 결론이 닿는다

개발을 할 때 모듈의 완전한 분리를 목표를 했지만 그것을 도와주는 모듈을 만들어 줬을 뿐 정작 코드를 보니 문제 1의 프로그램 보다는 다소 낫지만 스파게티 코드에 가깝다고 생각한다. 나중에 문제 3인 “죽음의 마블”을 만들 때는 특정 변수가 바뀔 때 그 변수에 대한 세터를 사용하는 WPF를 써서 좀 더 정교하고 유지보수하기 쉬운 게임을 만들어야겠다고 생각한다.

6. 자체 평가서

해당 문제를 수행하면서 가장 많이 느낀 것은 이런 미니 프로젝트 같은 경우 AI를 하루만 돌려도 문제가 해결되는 수준에 그치지만 우리는 AI가 생성한 코드는 전혀 사용하지 않고 순수하게 손으로 써야 한다는 목표가 존재 했기에 AI는 학습하는 것에만 한정하고 제작한 결과 무려 2주나 걸리게 되었다. 솔직하게 말하자면 중간고사도 껴 있기에 실제 제작 시간은 7일도 되지 않았지만 그래도 할애한 시간만큼 설계 실력이 한 층 늘어난 것 같아서 좋았다.

이번에 이 프로젝트를 수행하는 도중 모듈의 분리에 대한 필요성을 느껴 BindableEvent라는 기술을 사용하였는데, 이 기능은 독자적으로 생각한 기능은 아니다. 과거에 아무 지식도 없이 게임 만들기에 도전하고 싶어서 둘러본 게임 엔진에서 위 기능과 비슷한 기능이 있었는데, 이를 이용하여 다른 모듈을 불러오지 않고 독립적으로 데이터를 전달할 수 있었다. 저 로직이 떠올랐던 나는 이걸 파이썬으로 구현하고자 하였고 또 다시 설계의 늪에 빠져 하루동안 하드 코딩 하였다. 처음에는 AI의 도움으로 쉽게 구현에 성공하였지만 기분이 찜찜하였고 자고 일어나니 AI가 제안한 함수를 싹 다 갈아 엎을 정도로 좋은 아이디어가 떠올라서 코드를 싹 다 지우고 처음부터 작성하였다. 그랬더니 이 모듈을 구현한 지 3일이나 지나 있었다.

초기에 해당 모듈을 만들 때 기분이 찜찜하였던 이유는 이벤트 함수의 소유자를 찾는 과정이었다. 초반에 AI 에게 물어본 것은 해당 이벤트의 *Fire 함수를 사용한 모듈의 파일을 알아내서 그 파일이 아닌 모듈들의 필터, 콜백 함수를 실행하자는 것이었는데 실제로 이게 통했다. 다만 나의 생각은 달랐다. 멀티 쓰레드로 동작하는 경우 해당 모듈의 이름을 알지 못하는 상황이 벌어질까 생각했기 때문이다. 그래서 일단 하루 자고 나니 떠오른 것이 그냥 BindableEvent에 고유한 이름을 짓고, Connection 클래스를 만들어서 그 안에 이벤트 객체를 한데 모아 해당 이벤트의 이름이 아닌 이벤트만 이벤트 함수를 수행한다는 단순한 방식이었다. 이게 통할 것이라 생각한 이유는 단순히 이게 더 깔끔하고 작동하기가 편하다고 생각했기 때문이다. 또 모듈 그 자체를 인식한다는 것은 파일 시스템을 건드는 것이라 위험성이 너무 크다고 생각 하였기에 해당 모듈이 프로그램에 손상이 갈지도 모른다고 생각하였다. 지금 봐도 초기 이벤트 모듈은 싹 다 갈아 엎고 다시 만드는 게 나았을 것이다.

* inspect 라이브러리를 사용한 것으로 해당 라이브러리의 stack() 함수를 사용하여 현재부터 과거까지 실행된 코드의 흔적(호출 스택)이 담긴 리스트 내 객체의 frame(함수가 실행될 때 담긴 정보)변수에 inspect.getmodule(frame)을 사용하여 모듈을 찾는다.

7. 참고 문헌

[1] 김영학, 《PBL 방식으로 배우는 자료구조》, 홍릉과학출판사, 2020
[2] Roblox Corporation, <BindableFunction>
[https://create.roblox.com/docs/ko-kr/reference/engine/classes/BindableFunction]